

# Lecture 18: Record Organization Methods in Files

---

Amit Kumar Dhar

October 9, 2025

File Organization Methods

Metadata and Buffering

# **File Organization Methods**

---

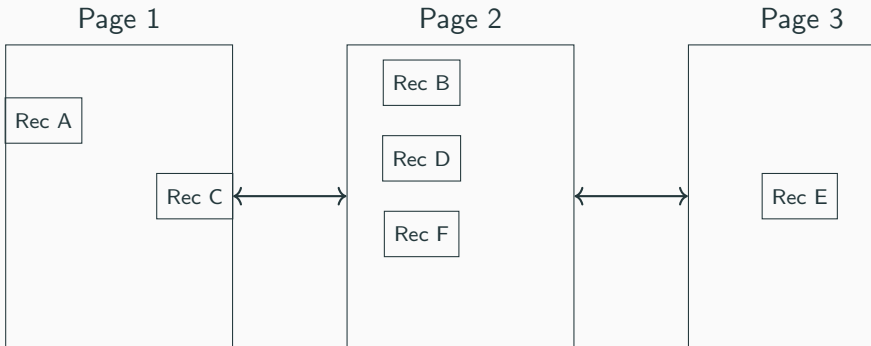
# Organization of Records in a File

How are the records (and pages) of a file structured?

1. **Heap File Organization:** Records are placed anywhere there is space.
2. **Sequential File Organization:** Records are stored in sorted order based on a "search key".
3. **Hashing File Organization:** A hash function is applied to a field to determine the page where the record should be placed.

# Heap File Organization

- Records are inserted at the end of the file, in the next available free space.
- Essentially an unordered pile of records.



# Heap File: Operations

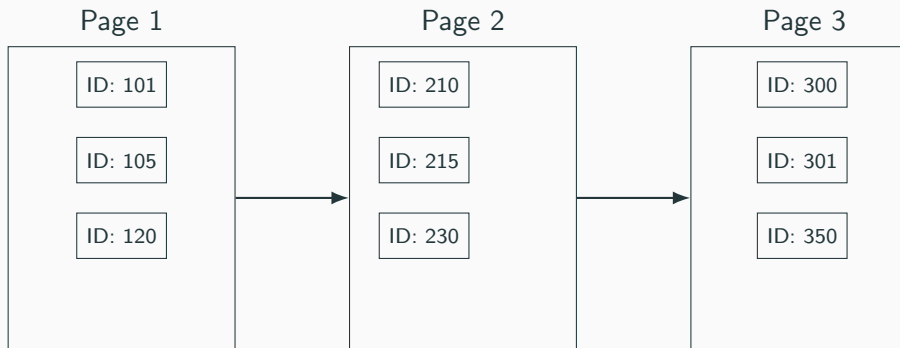
- **Insertion:** Very efficient. Find a page with free space and add the record.
- **Search:** Very inefficient. To find a record, you must perform a linear scan of all pages in the file.
- **Deletion:** Requires a search first. Once found, the record is removed.

## Best for...

Bulk loading data, or when searches on the data are rare.

# Sequential File Organization

- Records are physically ordered by the value of a **search key**.
- Example: Student records sorted by StudentID.



## Sequential File: Operations

- **Search:** Very efficient for key-based searches. Can use binary search on the pages to find the correct one quickly.
- **Insertion:** Inefficient. To maintain sort order, records may need to be shifted, and pages may need to be split. This can lead to a chain reaction of changes.
- **Deletion:** Also inefficient. Records are removed, and the remaining records may need to be compacted to reclaim space.

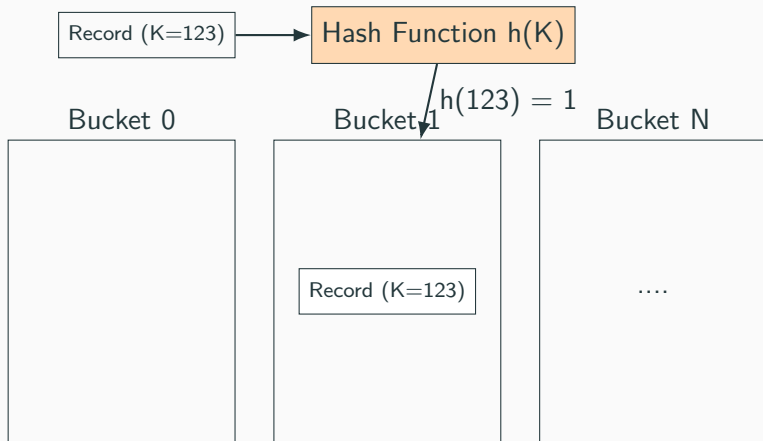
### Best for...

Applications with many queries but few updates, like data warehousing.



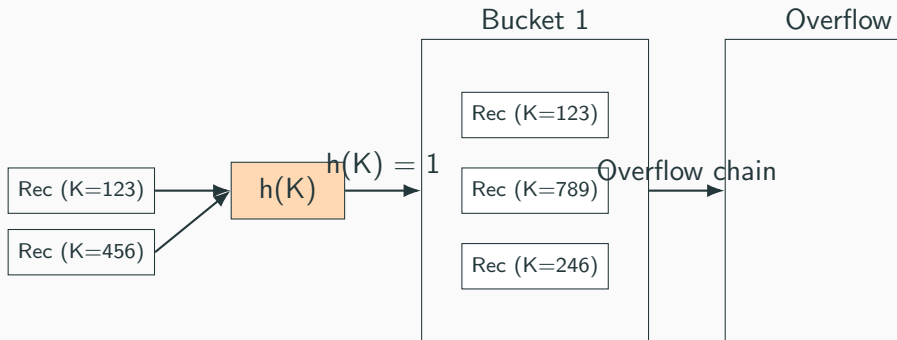
# Hashing File Organization

- A **hash function** is computed on a search key of a record.
- The result of the hash function,  $h(K)$ , specifies the page (or "bucket") where the record should be placed.



# Hashing: Collisions

- What if two keys hash to the same bucket? This is called a **collision**.
- **Solution:** Buckets can hold multiple records. If a bucket becomes full, we use **overflow pages** which are chained together in a linked list.



## Hashing File: Operations

- **Insertion:** Extremely fast. Compute hash, go to page, add record.
- **Search:** Extremely fast for exact key matches. Compute hash, go to page, find record.
- **Deletion:** Also very fast.
- **Range Queries:** Very inefficient. Hashing scrambles the order, so you cannot search for a range of keys (e.g., 'StudentID > 100 and < 200').

### Best for...

Applications that require very fast access to records based on an exact key value.

## Summary: How Methods Affect Operations

| Operation        | Heap             | Sequential       | Hashing          |
|------------------|------------------|------------------|------------------|
| Scan All Records | Good             | Good             | Bad              |
| Exact Key Search | Bad              | <b>Excellent</b> | <b>Excellent</b> |
| Range Search     | Bad              | <b>Excellent</b> | Bad              |
| Insertion        | <b>Excellent</b> | Bad              | <b>Excellent</b> |
| Deletion         | Bad              | Bad              | <b>Excellent</b> |

**Table 1:** Performance of file organizations for different operations.

**Key Takeaway:** The best file organization depends entirely on the expected workload.

## Demo: Storage Patterns of SQLite & PostgreSQL

# Metadata and Buffering

---

# Storage of Data Dictionary

- How does the DBMS know about tables, columns, types, indexes, etc.?
- This information is **metadata**.
- The metadata is stored in a **data dictionary** or **system catalog**.
- The data dictionary itself is stored as a set of tables!
  - E.g., a 'Tables' table, a 'Columns' table, an 'Indexes' table.
- The DBMS can query these catalog tables just like user tables to get information about the database structure.

# Buffering in DBMS

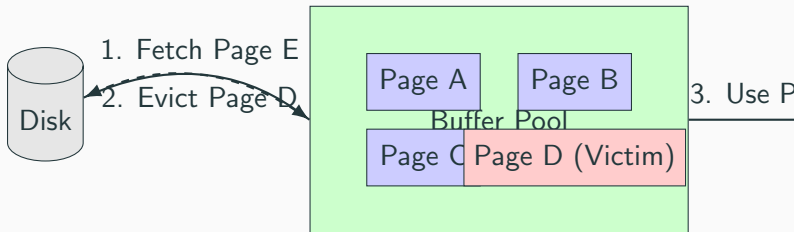
- Reading from disk is extremely slow compared to accessing main memory.
- The DBMS maintains a **buffer pool** in main memory, which is a collection of pages.
- When a process needs a page, it first checks the buffer pool.
  - **Buffer Hit:** The page is in the pool. Fast access!
  - **Buffer Miss:** The page is not in the pool. The DBMS must fetch it from disk, potentially replacing another page in the pool.
- The **buffer manager** is responsible for this process.



# Buffer Replacement Policy

- When the buffer pool is full and a new page is needed, which page should be replaced?
- This is governed by a **buffer replacement policy**.
- Common strategies:
  - **LRU (Least Recently Used)**: Replace the page that hasn't been accessed for the longest time. A very common and effective strategy.
  - **MRU (Most Recently Used)**: Replace the most recently used page. Useful for workloads with repeated scans of large tables.
  - **Clock Algorithm**: A more efficient approximation of LRU.

# Buffering Diagram



A buffer miss requires evicting a victim page (D) and loading the requested page (E).

**Questions?**